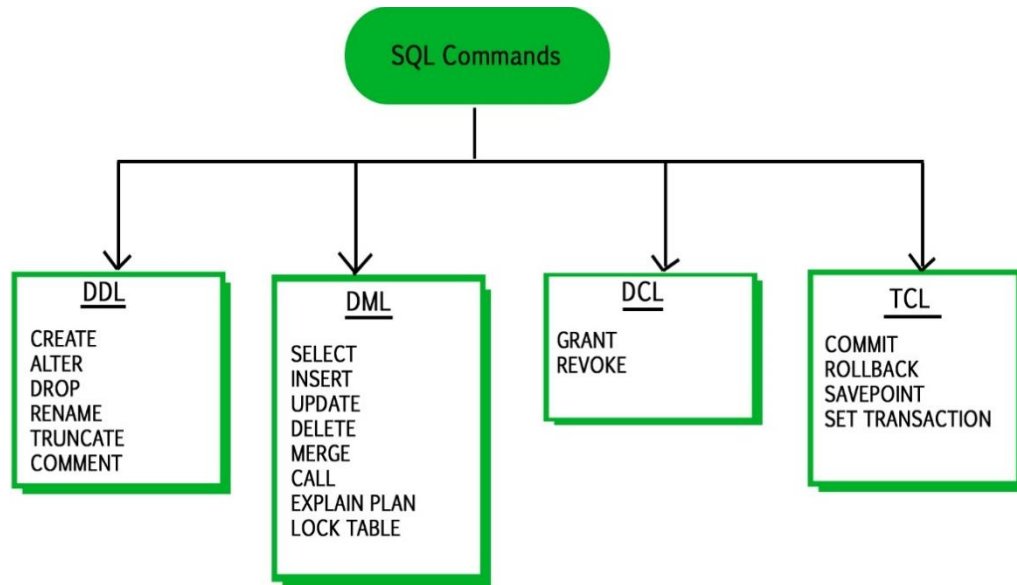


DDL, DML, TCL and DCL



Data Definition Language

- DDL is used to specify a database's structure, which includes its tables, views, indexes, and constraints.
- DDL commands come in the following types: CREATE, ALTER, DROP, RENAME, and TRUNCATE.
- DDL statements only modify the database's schema; they have no direct effect on the data within the database.
- DDL declarations are irreversible and difficult to undo.

Some Commands:

CREATE : to create objects in database
ALTER : alters the structure of database
DROP : delete objects from database
RENAME : rename an objects

Following SQL DDL-statement defines the department table :

```
create table department
(dept_name char(20),
building char(15),
budget numeric(12,2));
```

Execution of the above DDL statement creates the department table with three columns – dept_name, building, and budget; each of which has a specific datatype associated with it.

Data Manipulation Language

- Inserting, updating, removing, and retrieving data from a database are all possible with DML.
- DML commands come in the following types: SELECT, INSERT, UPDATE, DELETE, and MERGE.
- DML statements have a direct impact on the database's data.
- In the event of an error, data can be recovered thanks to the reversibility of DML statements.

DML statements are used for managing data with in schema objects.

DML are of two types –

Procedural DMLs : require a user to specify what data are needed and how to get those data.
Declarative DMLs (also referred as Non-procedural DMLs) : require a user to specify what data are needed without specifying how to get those data.

Declarative DMLs are usually easier to learn and use than procedural DMLs. However, since a user does not have to specify how to get the data, the database system has to figure out an efficient means of accessing data.

Some Commands :

SELECT: retrieve data from the database

INSERT: insert data into a table

UPDATE: update existing data within a table

DELETE: deletes all records from a table, space for the records remain

Example of SQL query that finds the names of all instructors in the History department :

```
select instructor.name from instructor  
where instructor.dept_name = 'History';
```

The query specifies that those rows from the table instructor where the dept_name is History must be retrieved and the name attributes of these rows must be displayed.

TCL (Transaction Control Language) :

Transaction Control Language commands are used to manage transactions in the database. These are used to manage the changes made by DML-statements. It also allows statements to be grouped together into logical transactions.

Examples of TCL commands –

COMMIT: Commit command is used to permanently save any transaction into the database.

ROLLBACK: This command restores the database to last committed state.
It is also used with savepoint command to jump to a savepoint in a transaction.

SAVEPOINT: Savepoint command is used to temporarily save a transaction so that you can rollback to that point whenever necessary.

DCL (Data Control Language) :

A Data Control Language is a syntax similar to a computer programming language used to control access to data stored in a database (Authorization). In particular, it is a component of Structured Query Language (SQL).

Examples of DCL commands :

GRANT: allow specified users to perform specified tasks.

REVOKE: cancel previously granted or denied permissions.

The operations for which privileges may be granted to or revoked from a user or role apply to both the Data definition language (DDL) and the Data manipulation language (DML), and may include CONNECT, SELECT, INSERT, UPDATE, DELETE, EXECUTE and USAGE.

DBMS Lab Manual

DDL	DML
It stands for <u>Data Definition Language</u> .	It stands for <u>Data Manipulation Language</u> .
It is used to create <u>database schema</u> and can be used to define some constraints as well.	It is used to add, retrieve, or update the data.
It basically defines the column (Attributes) of the table.	It adds or updates the row of the table. These rows are called tuples.
It doesn't have any further classification.	It is further classified into <u>Procedural and Non-Procedural</u> DML.
Basic commands present in DDL are CREATE, DROP, RENAME, ALTER, etc.	BASIC commands present in DML are UPDATE, INSERT, MERGE etc.
DDL does not use <u>WHERE clause</u> in its statement.	While DML uses <u>WHERE clause</u> in its statement.
DDL is used to define the structure of a database.	DML is used to manipulate the data within the database.
DDL is used to create and modify database objects like tables, indexes, views, and constraints.	DML is used to perform operations on the data within those database objects.
DDL statements are typically executed less frequently than DML statement	DML statements are frequently executed to manipulate and query data.
DDL statements are typically executed by database administrators.	DML statements are typically executed by application developers or end-users.
DDL statements are not used to manipulate data directly.	DML statements are used to manipulate data directly.
The database's contents are not altered by DDL commands.	DML commands alter the database's contents.
Examples of DDL commands: CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE TABLE, and RENAME TABLE.	Examples of DML commands: SELECT, INSERT, UPDATE, DELETE, and MERGE.

Data types

String Data Types

Data type	Description	Max size	Storage
char(n)	Fixed width character string	8,000 characters	Defined width
varchar(n)	Variable width character string	8,000 characters	2 bytes + number of chars
varchar(max)	Variable width character string	1,073,741,824 chars	2 bytes + number of chars
text	Variable width character string	2GB of text data	4 bytes + number of chars
nchar	Fixed width Unicode string	4,000 characters	Defined width x 2
nvarchar	Variable width Unicode string	4,000 characters	
nvarchar(max)	Variable width Unicode string	536,870,912 chars	
ntext	Variable width Unicode string	2GB of text data	
binary(n)	Fixed width binary string	8,000 bytes	
varbinary	Variable width binary string	8,000 bytes	
varbinary(max)	Variable width binary string	2GB	
image	Variable width binary string	2GB	

Numeric Data Types

Data type	Description	Storage
bit	Integer that can be 0, 1, or NULL	
tinyint	Allows whole numbers from 0 to 255	1 byte
smallint	Allows whole numbers between -32,768 and 32,767	2 bytes
int	Allows whole numbers between -2,147,483,648 and 2,147,483,647	4 bytes
bigint	Allows whole numbers between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807	8 bytes
decimal(p,s)	Fixed precision and scale numbers. Allows numbers from $-10^{38} + 1$ to $10^{38} - 1$. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes
numeric(p,s)	Fixed precision and scale numbers. Allows numbers from $-10^{38} + 1$ to $10^{38} - 1$. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes
smallmoney	Monetary data from -214,748.3648 to 214,748.3647	4 bytes
money	Monetary data from -922,337,203,685,477.5808 to 922,337,203,685,477.5807	8 bytes
float(n)	Floating precision number data from $-1.79E + 308$ to $1.79E + 308$. The n parameter indicates whether the field should hold 4 or 8 bytes. float (24) holds a 4-byte field and float (53) holds an 8-byte field. Default value of n is 53.	4 or 8 bytes
real	Floating precision number data from $-3.40E + 38$ to $3.40E + 38$	4 bytes

Date and time data types

- Date - Store a date only. From January 1, 0001 to December 31, 9999.
Format: yyyy-mm-dd
- Datetime - From January 1, 1753 to December 31, 9999 with an accuracy of 3.33 milliseconds. Format : yyyy-mm-dd hh:mi:ss
- Smalldatetime - From January 1, 1900 to June 6, 2079 with an accuracy of 1 minute. Format : yyyy-mm-dd hh:mi:ss
- Time- Store a time only to an accuracy of 100 nanoseconds

Note: the date types are chosen for a column when you create a new table in your database!

Other Data Types

Data type	Description
sql_variant	Stores up to 8,000 bytes of data of various data types, except text, ntext, and timestamp
uniqueidentifier	Stores a globally unique identifier (GUID)
xml	Stores XML formatted data. Maximum 2GB
cursor	Stores a reference to a cursor used for database operations
table	Stores a result-set for later processing

SQL CREATE Database

The SQL CREATE DATABASE statement is used to create new SQL database.

Syntax: **CREATE DATABASE DatabaseName;**

Always database name should be unique within the RDBMS.

Example:

```
CREATE DATABASE testDB;
```

DROP or DELETE Database

The SQL DROP DATABASE statement is used to drop an existing database in SQL schema.

Syntax:

```
Drop DATABASE DatabaseName;
```

Example:

```
Drop DATABASE testDB;
```

NOTE: Be careful before using this operation because by deleting an existing database would result in loss of complete information stored in the database.

Make sure you have admin privilege before dropping any database.

SQL CREATE Table

**Create table tablename(Column1 datatype, column2 datatype,
column datatype, primary key (one or more columns));**

DROP or DELETE Table

```
Drop table tablename;
```

INSERT data records on table

The SQL INSERT INTO Statement is used to add new rows of data to a table in the database.

Syntax:

```
Insert into tablename (column1, column2, ..., column)  
Values (data1, data2, ..., dataN);
```

View data record from table

SELECT Query

Select all data records from table

```
Select * from tablename;
```

Select only selected data records from selected column

```
Select column1, column2, ..., columnN from tablename;
```

ALTER TABLE Statement

The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.

To add a column in a table, use the following syntax:

ALTER TABLE table_name ADD column_name datatype

To delete a column in a table, use the following

ALTER TABLE table_name DROP COLUMN column_name

LAB TASK: 1

1. Create a database named 'ADVANCED' and create tables as following schema:
Student (SID,Name,RN,Batch)
Teacher (TID,Name, Faculty)
2. Insert any five records in each table
3. Display all records.
4. Display only ID and Name from student table.
5. Display Name and faculty from Teacher table.
6. Remove 'RN' attribute from student relation.
7. Add 'salary' attribute to teacher relation.
8. Copy ID and name attribute to new relation 'info_student'.
9. Delete all contents from info_student relation
10. Delete table name info_student.
11. Delete database that you created in Lab1.

Lab 2 Constraints, Integrity and Where clause

SQL Constraints:

Constraints are the rules enforced on data columns on table. These are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the database.

Constraints could be column level or table level. Column level constraints are applied only to one column, whereas table level constraints are applied to the whole table.

Following are commonly used constraints available in SQL:

NOT NULL Constraint: Ensures that a column cannot have NULL value.

DEFAULT Constraint: Provides a default value for a column when none is specified.

UNIQUE Constraint: Ensures that all values in a column are different.

PRIMARY Key: Uniquely identified each rows/records in a database table.

FOREIGN Key: Uniquely identified a rows/record in any another database table.

CHECK Constraint: The CHECK constraint ensures that all values in a column satisfy certain conditions.

INDEX: Use to create and retrieve data from the database very quickly.

NOT NULL Constraint:

By default, a column can hold NULL values. If you do not want a column to have a NULL value, then you need to define such constraint on this column specifying that NULL is now not allowed for that column.

A NULL is not the same as no data, rather, it represents unknown data.

```
create table Student (sid int not null, Name varchar(20) not null,  
RN int not null, Batch varchar(10) not null primary key (sid)  
);
```

DEFAULT Constraint:

The DEFAULT constraint provides a default value to a column when the INSERT INTO statement does not provide a specific value.

```
create table teachers (tid int not null, Name varchar(20) not null,  
salary decimal(10,2) DEFAULT 35000,  
faculty varchar(20) not null primary key (tid)  
);
```

Drop Default Constraint.

To drop DEFAULT constraint, use the following

```
Alter table column salary drop default;
```

UNIQUE Constraint:

The UNIQUE Constraint prevents two records from having identical values in a particular column. In the CUSTOMERS table, for example, you might want to prevent two or more people from having identical age.

```
create table Student (sid int not null, Name varchar(20) not null,  
RN int not null unique, Batch varchar(10) not null primary key (sid)  
);
```

You can also use following syntax, which supports naming the constraint in multiple columns as well

```
Alter table student add constraint myconstraint unique (rn, name);
```

Drop a unique constraint:

To drop a unique constraint. Using following SQL command:

Alter table student drop constraint myconstraint;

PRIMARY Key:

A primary key is a field in a table which uniquely identifies each row/record in a database table.

Primary keys must contain unique values. A primary key column cannot have NULL values.

A table can have only one primary key, which may consist of single or multiple fields. When multiple fields are used as a primary key, they are called a composite key.

If a table has a primary key defined on any field(s), then you cannot have two records having the same value of that field(s). **Note:** You would use these concepts while creating database tables.

Create Primary Key: syntax:

***create table customers (cid int not null, name varchar (20) not null,
age int not null, address char (25), salary decimal (18,2),
primary key(cid));***

For defining a primary key constraint on multiple columns, use the following syntax:

***create table customers (cid int not null, name varchar (20) not null,
age int not null, address char (25), salary decimal (18,2),
primary key(cid, name));***

To create a Primary Key constraint on the CID and Name columns when the CUSTOMERS table already exists, use the following command:

Alter table customers add constraint PKCustid primary key (cid, name);

Delete Primary key:

Alter table table name Drop constraint constraint_name;

FOREIGN Key:

A foreign key is a key used to link two tables together. This is sometimes called a referencing key. Foreign Key is a column or a combination of columns whose values match a Primary Key in a different table. The relationship between 2 tables matches the Primary Key in one of the tables with a Foreign Key in the second table. If a table has a primary key defined on any field(s), then you cannot have two records having the same value of that field(s).

Example: Consider the structure of the two tables as follows:

Customers table:

***create table customers (cid int not null, name varchar (20) not null,
age int not null, address char (25), salary decimal (18,2),
primary key(cid));***

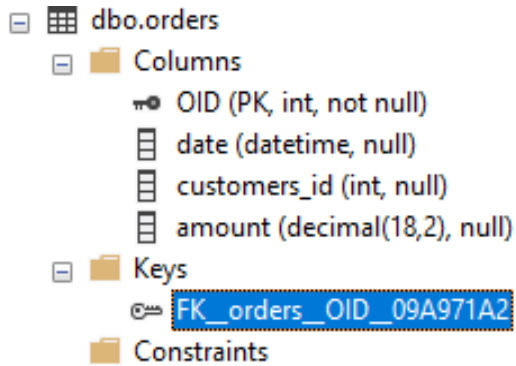
Order Table

***Create table orders
(OID int not null, date datetime, customers_id int references Customers
(CID), amount decimal (18,2), primary key (OID)
);***

Foreign key assign by

ALTER TABLE Orders

ADD FOREIGN KEY (OID) REFERENCES Customers(CID);



Drop a foreign key constraint

```
ALTER TABLE Orders
DROP constraint FK__orders__OID__09A971A2;
```

CHECK Constraint:

The CHECK Constraint enables a condition to check the value being entered into a record. If the condition evaluates to false, the record violates the constraint and isn't entered into the table.

```
create table customers (cid int not null, name varchar (20) not null,
age int not null check (age>=18), address char (25),
salary decimal (18,2), primary key(cid)
);
```

WHERE Clause

The SQL WHERE clause is used to specify a condition while fetching the data from single table or joining with multiple tables. If the given condition is satisfied, then only it returns specific value from the table. You would use WHERE clause to filter the records and fetching only necessary records. The WHERE clause is not only used in SELECT statement, but it is also used in UPDATE, DELETE statement, etc.,

Syntax:

```
SELECT column1, column2, columnN
FROM table_name
WHERE [condition]
```

You can specify a condition using comparison or logical operators.

Operators used in Where clause

Operator	Description	Example
=	Equal	Age=18
>	Greater than	Age>18
<	Less than	Age<18
>=	Greater than or equal	Age>=18
<=	Less than or equal	Age>18
<>	Not equal. Note: In some versions of SQL this operator may be written as !=	AgeN<>18
BETWEEN	Between a certain range	Age between 18 and 22

DBMS Lab Manual

LIKE	Search for a pattern using Percentage (%) and Underscore (_).	Name like 'S%' for starting with S. eg. Shaym, Sagar, Sita, Santosh, etc. Name like '%a' for ending with a. eg. Sita, Gita, Rita, Narendra, Dipendra, etc. Salary like '2__3' for salary start with 2 and ends with 3.
IN	To specify multiple possible values for a column	Address IN ('Ktm','PTN','BKT')

Arithmetic Operators

Operator	Description	Example
+	Add	5+2=7
-	Subtract	5-2=3
*	Multiply	5*2=10
/	Divide	5/2=2.5
%	Modulo	5%2=1

Compound Operators

Operator	Description
+=	Add equals
-=	Subtract equals
*=	Multiply equals
/=	Divide equals
%=	Modulo equals
&=	Bitwise AND equals
^-=	Bitwise exclusive equals
=	Bitwise OR equals

SQL Logical Operators

Operator	Description
ALL	TRUE if all of the subquery values meet the condition
AND	TRUE if all the conditions separated by AND is TRUE
ANY	TRUE if any of the subquery values meet the condition
BETWEEN	TRUE if the operand is within the range of comparisons
EXISTS	TRUE if the subquery returns one or more records
IN	TRUE if the operand is equal to one of a list of expressions
LIKE	TRUE if the operand matches a pattern
NOT	Displays a record if the condition(s) is NOT TRUE
OR	TRUE if any of the conditions separated by OR is TRUE
SOME	TRUE if any of the subquery values meet the condition

AND and OR Operators

The AND Operator: The AND operator allows the existence of multiple conditions in an SQL statement's WHERE clause.

Syntax:

The basic syntax of AND operator with WHERE clause is as follows:

```
SELECT column1, column2, columnN FROM table_name
WHERE [condition1] AND [condition2]...AND [conditionN];
```

The OR Operator: The OR operator is used to combine multiple conditions in an SQL statement's WHERE clause.

Syntax:

```
SELECT column1, column2, columnN FROM table_name
WHERE [condition1] OR [condition2]...OR [conditionN]
```

BETWEEN Operator

The BETWEEN operator selects values within a given range. The values can be numbers, text, or dates. The BETWEEN operator is inclusive: begin and end values are included.

Example

Selects all products with a price between 10 and 20:

```
select * from products where price between 10 and 20;
```

NOT BETWEEN

To display the products outside the range of the previous example, use NOT BETWEEN:

Example

```
select * from products
where price not between 10 and 20;
```

BETWEEN with IN

The following SQL statement selects all products with a price between 10 and 20. In addition, the CategoryID must be either 1,2, or 3:

Example

```
select * from products
where price between 10 and 20
and categoryid in (1,2,3);
```

BETWEEN Text Values

The following SQL statement selects all products with a ProductName alphabetically between Carnarvon Tigers and Mozzarella di Giovanni:

Example

```
select * from products
where productname between 'carnarvon tigers' and 'mozzarella di giovanni'
order by productname;
```

The following SQL statement selects all products with a ProductName between Carnarvon Tigers and Chef Anton's Cajun Seasoning:

Example

```
select * from products
where productname between "carnarvon tigers" and "chef anton's cajun seasoning"
order by productname;
```

NOT BETWEEN Text Values

The following SQL statement selects all products with a ProductName not between Carnarvon Tigers and Mozzarella di Giovanni:

Example

```
select * from products
where productname not between 'carnarvon tigers' and 'mozzarella di giovanni'
order by productname;
```

BETWEEN Dates

The following SQL statement selects all orders with an OrderDate between '01-July-1996' and '31-July-1996':

Example

```
select * from orders
where orderdate between #07/01/1996# and #07/31/1996#;
```

OR:

Example

```
select * from orders
where orderdate between '1996-07-01' and '1996-07-31';
```

IN Operator

The IN operator allows you to specify multiple values in a WHERE clause. The IN operator is a shorthand for multiple OR conditions.

Example Return all customers from 'Germany', 'France', or 'UK'

```
select * from customers
where country in ('germany', 'france', 'uk');
```

Syntax

```
select column_name(s)
from table_name
where column_name in (value1, value2, ...);
```

NOT IN

By using the NOT keyword in front of the IN operator, you return all records that are NOT any of the values in the list.

Example Return all customers that are NOT from 'Germany', 'France', or 'UK':

```
select * from customers
where country not in ('germany', 'france', 'uk');
```

IN (SELECT)

You can also use IN with a subquery in the WHERE clause. With a subquery you can return all records from the main query that are present in the result of the subquery.

Example

Return all customers that have an order in the **Orders** table:

```
SELECT * FROM Customers
WHERE CustomerID IN (SELECT CustomerID FROM Orders);
```

NOT IN (SELECT)

The result in the example above returned 74 records, that means that there are 17 customers that haven't placed any orders. Let us check if that is correct, by using the NOT IN operator.

Example

Return all customers that have NOT placed any orders in the **Orders** table:

```
select * from customers
where customerid not in (select customerid from orders);
```

The SQL ANY and ALL Operators

The ANY and ALL operators allow you to perform a comparison between a single column value and a range of other values.

The ANY operator: returns a boolean value as a result, returns TRUE if ANY of the subquery values meet the condition, ANY means that the condition will be true if the operation is true for any of the values in the range.

ANY Syntax

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ANY
(SELECT column_name
FROM table_name
WHERE condition);
```

Note: The *operator* must be a standard comparison operator (=, <>, !=, >, >=, <, or <=).

The ALL operator: ALL means that the condition will be true only if the operation is true for all values in the range. returns a boolean value as a result, returns TRUE if ALL of the subquery values meet the condition, is used with SELECT, WHERE and HAVING statements

ALL Syntax With SELECT

```
SELECT ALL column_name(s)
FROM table_name
WHERE condition;
```

ALL Syntax With WHERE or HAVING

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ALL
(SELECT column_name
FROM table_name
WHERE condition);
```

Note: The *operator* must be a standard comparison operator (=, <>, !=, >, >=, <, or <=).

SQL ANY Examples

The following SQL statement lists the ProductName if it finds ANY records in the OrderDetails table has Quantity equal to 10 (this will return TRUE because the Quantity column has some values of 10):

Example

```
select productname
from products
where productid = any
(select productid
from orderdetails
where quantity = 10);
```

The following SQL statement lists the ProductName if it finds ANY records in the OrderDetails table has Quantity larger than 99 (this will return TRUE because the Quantity column has some values larger than 99):

Example

```
select productname
from products
where productid = any
(select productid
from orderdetails
where quantity > 99);
```

The following SQL statement lists the ProductName if it finds ANY records in the OrderDetails table has Quantity larger than 1000 (this will return FALSE because the Quantity column has no values larger than 1000):

Example

```
select productname
from products
where productid = any
(select productid
from orderdetails
where quantity > 1000);
```

SQL ALL Examples The following SQL statement lists ALL the product names:

Example

```
select all productname
from products
where true;
```

The following SQL statement lists the ProductName if ALL the records in the OrderDetails table has Quantity equal to 10. This will of course return FALSE because the Quantity column has many different values (not only the value of 10):

Example

```
select productname
from products
where productid = all
(select productid
from orderdetails
where quantity = 10);
```

SQL SOME Statement:

```
select * from book
where price > some (select price from book where price > 400);
```

EXISTS Operator: The EXISTS operator is used to test for the existence of any record in a subquery, returns TRUE if the subquery returns one or more records.

EXISTS Syntax

```
select column_name(s) from table_name
where exists (select column_name from table_name where condition);
```

Demo Database

Below is a selection from the "Products" table in the Northwind sample database:
And a selection from the "Suppliers" table:

Supplier ID	Supplier Name	Contact Name	Address	City	Postal Code	Country
1	Exotic Liquid	Charlotte Cooper	49 Gilbert St.	London	EC1 4SD	UK
2	New Orleans Cajun Delights	Shelley Burke	P.O. Box 78934	New Orleans	70117	USA
3	Grandma Kelly's Homestead	Regina Murphy	707 Oxford Rd.	Ann Arbor	48104	USA
4	Tokyo Traders	Yoshi Nagase	9-8 Sekimai Musashino-shi	Tokyo	100	Japan

SQL EXISTS Examples : The following SQL statement returns TRUE and lists the suppliers with a product price less than 20:

Example

```
select suppliername from suppliers
where exists (select productname from products where products.supplierid =
suppliers.supplierid and price < 20);
```

The following SQL statement returns TRUE and lists the suppliers with a product price equal to 22:

Example

```
select suppliername from suppliers
where exists (select productname from products where products.supplierid =
suppliers.supplierid and price = 22);
```

SQL Expressions

An expression is a combination of one or more values, operators, and SQL functions that evaluate to a value. SQL EXPRESSIONS are like formulas and they are written in query language. You can also use them to query the database for specific set of data.

Syntax:

```
SELECT column1, column2, columnN FROM table_name
WHERE [CONDITION|EXPRESSION];
```

Example 1:

```
SELECT COUNT(*) AS "RECORDS" FROM book;
```

Output:

	RECORDS
1	4

Example 2:

```
SELECT BID, Price,
CASE
WHEN price > 500 THEN 'The Price is greater than 500'
WHEN price = 500 THEN 'The price is 500'
ELSE 'The Price is under 500'
END AS Remarks
FROM book;
```

Output:

	BID	price	Remarks
1	2001	500	The price is 500
2	2002	295	The Price is under 500
3	2003	450	The Price is under 500
4	2004	600	The Price is greater than 500

AUTO INCREMENT Field

Auto-increment allows a unique number to be generated automatically when a new record is inserted into a table. Often this is the primary key field that we would like to be created automatically every time a new record is inserted.

Syntax for SQL Server

The following SQL statement defines the "Personid" column to be an auto-increment primary key field in the "Persons" table:

```
CREATE TABLE Persons (
    Personid int IDENTITY(1,1) PRIMARY KEY,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int
);
select * from Persons;
Personid  LastName  FirstName  Age
```

The SQL Server uses the IDENTITY keyword to perform an auto-increment feature. In the example above, the starting value for IDENTITY is 1, and it will increment by 1 for each new record.

Tip: To specify that the "Personid" column should start at value 10 and increment by 5, change it to IDENTITY(10,5). To insert a new record into the "Persons" table, we will NOT have to specify a value for the "Personid" column (a unique value will be added automatically):

```
INSERT INTO Persons (FirstName,LastName, Age)
VALUES ('Shyam','Acharya',55),
       ('Sangita', 'Sharma',45),
       ('Pankaj', 'Pradhan',44),
       ('Rima', 'Gurung',33),
       ('Sita','Lama',32);
```

```
select * from Persons;
```

	Personid	LastName	FirstName	Age
1	1	Acharya	Shyam	55
2	2	Sharma	Sangita	45
3	3	Pradhan	Pankaj	44
4	4	Gurung	Rima	33
5	5	Lama	Sita	32

BACKUP DATABASE Statement: The BACKUP DATABASE statement is used in SQL Server to create a full back up of an existing SQL database.

Syntax

```
BACKUP DATABASE databasename
TO DISK = 'filepath';
```

Example: The following SQL statement creates a full back up of the existing database "testDB" to the D disk: Always back up the database to a different drive than the actual database. Then, if you get a disk crash, you will not lose your backup file along with the database.

```
BACKUP DATABASE shyam
TO DISK = 'D:\back\testdiff.bak';
```

BACKUP WITH DIFFERENTIAL Statement: A differential back up only backs up the parts of the database that have changed since the last full database backup.

Syntax

```
BACKUP DATABASE databasename
TO DISK = 'filepath'
WITH DIFFERENTIAL;
```

Example: A differential back up reduces the back up time (since only the changes are backed up).

```
BACKUP DATABASE shyam
TO DISK = 'D:\back\testdiff.bak'
WITH DIFFERENTIAL;
```

LAB TASK:2

- Create the relations of tables as given below:
 - Student (sid, Name , RN , Batch)
 - Teacher (tid, ename, dateofjoin, salary)
 - Employee (eid, ename, dateofemployee, salary)
 - Booklist (isbn, name, publication)
 - Book(bid, bname, author, price)
 - Issues(IID,name,dateofissue)
- Modify relation teacher and student
 - Set Tid as foreign key
 - Set SID as primary key
 - Delete RN attribute.
- Set default value of 'dateofemployee' attribute as Jan 1, 2010.
- Assign Bid and iid as foreign key.
- Specify the data entry criteria on price of books must be less than 5000.
- Ename, bname, name attribute of each relation must contain some value.
- Insert any 4 records in each relation.
- Display all records from all relations.
- Display eid and ename of all employees whose salary is less than 30000.
- Display all record of book whose price ranges from 400 to 600.
- Display all the records from booklist relation whose publication name starts from 'E' eg Ekta.
- Display all records from employee table whose name ends with 'a' eg Sita, Geeta etc.
- Display sid and name from student table whose name exactly consist of 5 characters.
- Display all records from employee table where name starts with 'P' and salary greater than 25000.
- Display all records from book table where either bid lies in range 2003 to 2004 or price range in 350 to 600.
- Display isbn number and bookname where booklist must not contain isbn no. 2003

Lab 3 Update Query, Delete query & Sub Query

UPDATE Query

The SQL UPDATE Query is used to modify the existing records in a table. You can use WHERE clause with UPDATE query to update selected rows, otherwise all the rows would be affected.

Syntax:

```
UPDATE table_name SET column1 = value1, column2 = value2..., columnN = valueN
WHERE [condition];
```

You can combine N number of conditions using AND or OR operators.

Example:

```
select * from issues;
      update issues set bid=iid+100;
select * from issues;

select * from teachers;
      update teachers set salary=salary+salary*.1;
select * from teachers;
```

DELETE Query

The SQL DELETE Query is used to delete the existing records from a table. You can use WHERE clause with DELETE query to delete selected rows, otherwise all the records would be deleted.

Syntax:

```
Delete from table_name where [condition];
```

You can combine N number of conditions using AND or OR operators.

Example:

```
Delete from customers where id = 6;
```

SQL TRUNCATE TABLE

The SQL TRUNCATE TABLE command is used to delete complete data from an existing table. You can also use DROP TABLE command to delete complete table but it would remove complete table structure from the database and you would need to re-create this table once again if you wish you store some data.

Syntax:

The basic syntax of TRUNCATE TABLE is as follows:

```
truncate table table_name;
```

Example:

```
Truncate Table Customers
```

Output:

```
SELECT * FROM CUSTOMERS;
```

SQL Sub Queries

- A Sub query or Inner query or Nested query is a query within another SQL query and embedded within the WHERE clause.
- A sub query is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.
- Sub queries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN etc.
- There are a few rules that sub queries must follow:
 - i. Subqueries must be enclosed within parentheses.
 - ii. A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.

- iii. An ORDER BY cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY can be used to perform the same function as the ORDER BY in a subquery.
- iv. Subqueries that return more than one row can only be used with multiple value operators, such as the IN operator.
- v. The SELECT list cannot include any references to values that evaluate to a BLOB, ARRAY, CLOB, or NCLOB.
- vi. A subquery cannot be immediately enclosed in a set function.
- vii. The BETWEEN operator cannot be used with a subquery; however, the BETWEEN operator can be used within the subquery.

Subqueries with the SELECT Statement:

Subqueries are most frequently used with the SELECT statement. The basic syntax is as follows:

Syntax:

```
SELECT column_name [, column_name ] FROM table1 [, table2 ]
WHERE column_name OPERATOR
(SELECT column_name [, column_name ]
FROM table1 [, table2 ] [WHERE] condition);
```

Example:

```
SELECT * FROM CUSTOMERS
WHERE ID IN (SELECT ID FROM CUSTOMERS
WHERE SALARY > 4500) ;
```

Subqueries with the INSERT Statement:

Subqueries also can be used with INSERT statements. The INSERT statement uses the data returned from the subquery to insert into another table. The selected data in the subquery can be modified with any of the character, date or number functions.

Syntax:

```
INSERT INTO table_name [ (column1 [, column2 ]) ]
SELECT [ *|column1 [, column2 ] FROM table1 [, table2 ]
[ WHERE VALUE OPERATOR ]
```

Example:

```
INSERT INTO CUSTOMERS
SELECT * FROM CUSTOMERS WHERE ID IN (SELECT ID FROM CUSTOMERS) ;
```

Subqueries with the UPDATE Statement:

The subquery can be used in conjunction with the UPDATE statement. Either single or multiple columns in a table can be updated when using a subquery with the UPDATE statement. The basic syntax is as follows:

Syntax:

```
UPDATE table SET column_name = new_value [ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME FROM TABLE_NAME)
[ WHERE VALUE OPERATOR]];
```

Example: Update SALARY by 0.25 times in CUSTOMERS table for all the customers whose AGE is greater than or equal to 27:

```
UPDATE CUSTOMERS SET SALARY = SALARY * 0.25
WHERE AGE IN (SELECT AGE FROM CUSTOMERS WHERE AGE >= 27) ;
```

Subqueries with the DELETE Statement:

The subquery can be used in conjunction with the DELETE statement like with any other statements mentioned above. The basic syntax is as follows:

Syntax:

```
DELETE FROM TABLE_NAME
[ WHERE OPERATOR [ VALUE ]
(SELECT COLUMN_NAME FROM TABLE_NAME)
[ WHERE VALUE OPERATOR ]];
```

Example:

Assuming, we have CUSTOMERS_BKP table available which is backup of CUSTOMERS table. Following example deletes records from CUSTOMERS table for all the customers whose AGE is greater than or equal to 27:

```
DELETE FROM CUSTOMERS
WHERE AGE IN (SELECT AGE FROM CUSTOMERS_BKP WHERE AGE > 27 );
```

select * from Persons

	Personid	LastName	FirstName	Age
1	1	Chandra	Smair	23
2	2	Shama	Naresh	24
3	3	Pradhan	Arun	30

SELECT Personid, FirstName + ' ' + LastName as Fullname , Age FROM Persons;

	Personid	Fullname	Age
1	1	Smair Chandra	23
2	2	Naresh Shama	24
3	3	Arun Pradhan	30

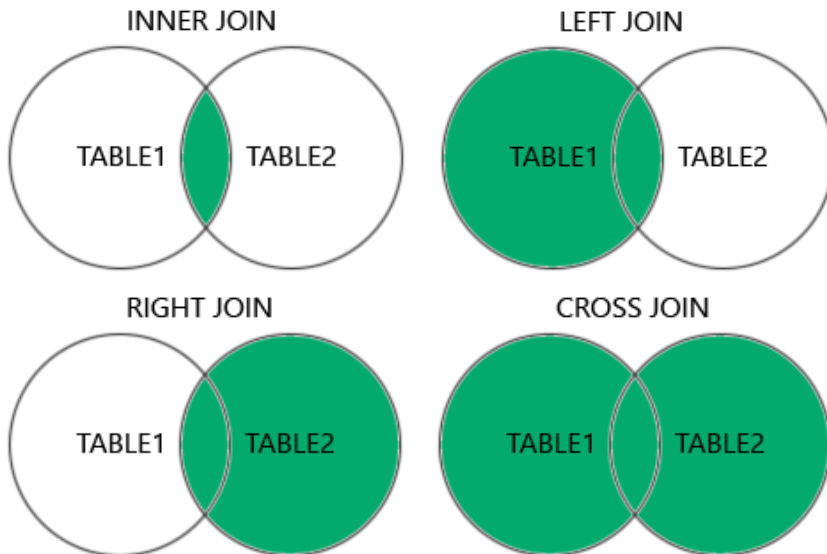
Lab Task 3

- Find all the bookname, publication name and author name where publication name is "Ekta".
- Find the teachers name and faculty who issued book on 2080 Baishak 21.
- Find the employee's name whose salary is greater than 25000 and faculty is "computer".
- Add attribute bid on Issues relation.
- Insert the data in bid column.
- Find the Teacher's name, and book name issued by the teacher whose name starts with "C".
- Update all salary by 10 %.
- Update book name SQL Programming as DATABASE.
- Update the salary of all employee by 20% whose salary is less than 26000.
- Provide 5% increment to all salaries whose salary is greater than 20000 and 20% increment in rest of salaries. (use CASE WHEN <CONDITION>THEN <STATEMENT> ELSE <STATEMENT> END).
- Delete the records from employee table whose eid is 111.
- Use sub query to find all teachers name and faculty whose date of employee is 2016-05-25.
- Use sub query to find all the book name and author name whose publication is "Ekta".

Lab 4

Introduction to Joins & Built in Functions in SQL, Group by clause, Having clause

Types of Joins



INNER JOIN: Returns records that have matching values in both tables

Syntax:

```
SELECT column_name(s) FROM table1 INNER JOIN table2
ON table1.column_name = table2.column_name;
```

Example:

```
SELECT bname,price FROM Book INNER JOIN Booklist ON Book.bid= booklist.isbn;
```

Output:

	bname	price
1	DBMS	500
2	C++ Programming	295
3	Computer Fundamentals	450
4	Data Mining	600

LEFT JOIN: Returns all records from the left table, and the matched records from the right table

Syntax:

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

Example:

```
SELECT * FROM Book
left JOIN Booklist
ON book.author = booklist.publication;
```

Output

	bid	bname	author	price	bbid	isbn	name	publication
1	2001	DBMS	SMA	500	2201	NULL	NULL	NULL
2	2002	C++ Programming	SMA	295	2202	NULL	NULL	NULL
3	2003	Computer Fundamentals	Ekta	450	2203	2003	Computer Fundamentals	Ekta
4	2003	Computer Fundamentals	Ekta	450	2203	2004	Data Mining	Ekta
5	2004	Data Mining	Ekta	600	2204	2003	Computer Fundamentals	Ekta
6	2004	Data Mining	Ekta	600	2204	2004	Data Mining	Ekta

RIGHT JOIN: Returns all records from the right table, and the matched records from the left table

Syntax:

```
SELECT column_name(s) FROM table1 RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

Example:

```
SELECT * FROM Book right JOIN Booklist ON book.author = booklist.publication;
```

Output:

	bid	bname	author	price	bbid	isbn	name	publication
1	NULL	NULL	NULL	NULL	NULL	2001	DBMS	SMA Publication
2	NULL	NULL	NULL	NULL	NULL	2002	C++ Programming	SMA Publication
3	2003	Computer Fundamentals	Ekta	450	2203	2003	Computer Fundamentals	Ekta
4	2004	Data Mining	Ekta	600	2204	2003	Computer Fundamentals	Ekta
5	2003	Computer Fundamentals	Ekta	450	2203	2004	Data Mining	Ekta
6	2004	Data Mining	Ekta	600	2204	2004	Data Mining	Ekta

CARTESIAN JOIN

The CARTESIAN JOIN or CROSS JOIN returns the cartesian product of the sets of records from the two or more joined tables. Thus, it equates to an inner join where the join-condition always evaluates to True or where the join condition is absent from the statement.

Syntax:

```
SELECT table1.column1, table2.column2...
FROM table1, table2 [, table3 ]
```

Example:

```
SELECT ID, NAME, AMOUNT, DATE
FROM CUSTOMERS, ORDERS;
```

Syntax:

```
SELECT column_name(s) FROM table1 CROSS JOIN table2;
```

Example1:

```
SELECT Book.bname, booklist.name FROM Book
CROSS JOIN Booklist WHERE Book.bid=booklist.isbn;
```

Output1:

	bname	name
1	DBMS	DBMS
2	C++ Programming	C++ Programming
3	Computer Fundamentals	Computer Fundamentals
4	Data Mining	Data Mining

Example2:

```
SELECT * FROM Book CROSS JOIN Booklist WHERE Book.bid=booklist.isbn;
```

Output2:

	bid	bname	author	price	bbid	isbn	name	publication
1	2001	DBMS	SMA	500	2201	2001	DBMS	SMA Publication
2	2002	C++ Programming	SMA	295	2202	2002	C++ Programming	SMA Publication
3	2003	Computer Fundamentals	Ekta	450	2203	2003	Computer Fundamentals	Ekta
4	2004	Data Mining	Ekta	600	2204	2004	Data Mining	Ekta

Group By clause:

The SQL GROUP BY clause is used in collaboration with the SELECT statement to arrange identical data into groups. The GROUP BY clause follows the WHERE clause in a SELECT statement and precedes the ORDER BY clause.

Syntax: The basic syntax of GROUP BY clause is given below. The GROUP BY clause must follow the conditions in the WHERE clause and must precede the ORDER BY clause if one is used.

```
Select column1, column2
From table_name
Where [ conditions]
Group by column1, column2 order by column1, column2
```

Example:

```
Select name, sum(salary) from customers
Group by name;
```

ORDER BY Clause

The SQL ORDER BY clause is used to sort the data in ascending or descending order, based on one or more columns. Some database sorts query results in ascending order by default.

Syntax:

```
Select column-list From table_name
[where condition]
[order by column1, column2, .. Columnn] [asc | desc];
```

You can use more than one column in the ORDER BY clause. Make sure whatever column you are using to sort, that column should be in column-list.

Example:

```
Select * from customers Order by name desc;
```

TOP Clause

The SQL TOP clause is used to fetch a TOP N number or X percent records from a table. Note: All the databases do not support TOP clause. For example MySQL supports LIMIT clause to fetch limited number of records and Oracle uses ROWNUM to fetch limited number of records.

Syntax:

```
Select top number|percent column_name(s)
From table_name where [condition]
```

Example:

```
Select top 3 * from customers;
```

HAVING CLAUSE

The HAVING clause enables you to specify conditions that filter which group results appear in the final results. The WHERE clause places conditions on the selected columns, whereas the HAVING clause places conditions on groups created by the GROUP BY clause. The HAVING clause must follow the GROUP BY clause in a query and must also precede the ORDER BY clause if used. The following is the syntax of the SELECT statement, including the HAVING clause:

Syntax:

```
select column1, column2 from table1, table2
where [ conditions ]
group by column1, column2
having [ conditions ] order by column1, column2
```

Example: display record for which similar age count would be more than or equal to 2:

```
select * from customers
```

group by age
having count(age) >= 2;

Built-in functions

SQL has many built-in functions for performing processing on string or numeric data. Following is the list of all useful SQL built-in functions:

COUNT Function - The SQL COUNT aggregate function is used to count the number of rows in a database table.

MAX Function - The SQL MAX aggregate function allows us to select the highest (maximum) value for a certain column.

MIN Function - The SQL MIN aggregate function allows us to select the lowest (minimum) value for a certain column.

AVG Function - The SQL AVG aggregate function selects the average value for certain table column.

SUM Function - The SQL SUM aggregate function allows selecting the total for a numeric column.

SQRT Functions - This is used to generate a square root of a given number.

RAND Function - This is used to generate a random number using SQL command.

CONCAT Function - This is used to concatenate any string inside any SQL command.

Numeric Functions - Complete list of SQL functions required to manipulate numbers in SQL.

String Functions - Complete list of SQL functions required to manipulate strings in SQL.

COUNT Function :COUNT function is the simplest function and very useful in counting the number of records , which are expected to be returned by a SELECT statement.

Example:

select count(*) from employee ;

Similarly, if you want to count the number of records for Nepal, then it can be done as follows:

select count(*) from employee where country= 'nepal';

MAX Function : MAX function is used to find out the record with maximum value among a record set.

Example:

select * from Persons

output:

	Personid	LastName	FirstName	Age
1	1	Chandra	Smair	23
2	2	Shama	Naresh	24
3	3	Pradhan	Arun	30

select max(age)as MaximumAge from Persons;

output:

	MaximumAge
1	30

MIN Function : MIN function is used to find out the record with minimum value among a record set.

select min(age) as MinimumAge from Persons;

	MinimumAge
1	23

AVG Function :AVG function is used to find out the average of a field in various records.

select avg(age)as Average_Age from Persons;

	Average_Age
1	25

SUM Function :SUM function is used to find out the sum of a field in various records.

```
select sum(age)as Sum_Of_Age from Persons;
```

	Sum_Of_Age
1	77

SQRT Function :SQRT function is used to find out the square root of any number. You can Use SELECT statement to find out square root of any number.

```
select sqrt(25) as SQRT_of_25;
```

	SQRT_of_25
1	5

ABS(X)

The ABS() function returns the absolute value of X.

```
select abs(-25) as 'Absolute_of_-25';
```

	Absolute_of_-25
1	25

Lab Task: 4

1. Sort the employee records in descending order.
2. Sort name and publication name in ascending order.
3. Display top three records from teachers relation.
4. Display the sum of salaries of all employees.
5. Display the minimum salary of employee.
6. Display the average price of book written by same author.
7. Display publication name and number of books published by it from book list relation publication wise.
8. Display the bid and bname of books whose price is greater than average prices of book.
9. Find the bid, bname and author in ascending order where author name is started by "s".
10. Find the teachers name and book taken by him. The teacher's salary who takes the book should be the maximum salary.
11. Find the authors name who have written more than one book.

Union & Creating Views

The SQL UNION Operator

The UNION operator is used to combine the result-set of two or more SELECT statements.

- Every SELECT statement within UNION must have the same number of columns
- The columns must also have similar data types
- The columns in every SELECT statement must also be in the same order

UNION Syntax

```
select column_name(s) from table1
union
select column_name(s) from table2;
```

UNION ALL Syntax : The UNION operator selects only distinct values by default. To allow duplicate values, use UNION ALL:

```
select column_name(s) from table1
union all
select column_name(s) from table2;
```

Note: The column names in the result-set are usually equal to the column names in the first SELECT statement.

Example

```
select city from customers
union
select city from suppliers
order by city;
```

Note: If some customers or suppliers have the same city, each city will only be listed once, because UNION selects only distinct values. Use UNION ALL to also select duplicate values!

SQL UNION ALL Example: The following SQL statement returns the cities (duplicate values also) from both the "Customers" and the "Suppliers" table:

Example

```
select city from customers
union all
select city from suppliers
order by city;
```

SQL UNION WITH WHERE: The following SQL statement returns the German cities (only distinct values) from both the "Customers" and the "Suppliers" table:

Example

```
select city, country from customers where country='germany'
union
select city, country from suppliers where country='germany'
order by city;
```

SQL UNION ALL With WHERE: The following SQL statement returns the German cities (duplicate values also) from both the "Customers" and the "Suppliers" table:

Example

```
select city, country from customers
where country='germany' union all
select city, country from suppliers
where country='germany' order by city;
```

Another UNION Example:**Example**

```
SELECT 'Customer' AS Type, ContactName, City, Country FROM Customers
UNION
SELECT 'Supplier', ContactName, City, Country FROM Suppliers;
```

SQL UNION

The following SQL statement returns the only distinct values from both the tables.

```
SELECT 'Book' AS Type, bName, author, price FROM Book
UNION
SELECT 'Publication', Publication, name, isbn FROM Booklist;
```

	Type	bName	author	price
1	Book	C++ Programming	SMA	295
2	Book	Computer Fundamentals	Ekta	450
3	Book	Data Mining	Ekta	600
4	Book	DBMS	SMA	500
5	Publication	SMA Publication	DBMS	2001
6	Publication	SMA Publication	C++ Programming	2002
7	Publication	Ekta	Computer Fundamentals	2003
8	Publication	Ekta	Data Mining	2004

SQL - Using Views

A view is nothing more than a SQL statement that is stored in the database with an associated name. A view is actually a composition of a table in the form of a predefined SQL query. A view can contain all rows of a table or select rows from a table. A view can be created from one or many tables which depends on the written SQL query to create a view.

Views, which are kind of virtual tables, allow users to do the following:

- Structure data in a way that users or classes of users find natural or intuitive.
- Restrict access to the data such that a user can see and (sometimes) modify exactly what they need and no more.
- Summarize data from various tables which can be used to generate reports.

Creating Views:

Database views are created using the CREATE VIEW statement. Views can be created from a single table, multiple tables, or another view. To create a view, a user must have the appropriate system privilege according to the specific implementation.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2.....
FROM table_name
WHERE [condition];
```

You can include multiple tables in your SELECT statement in very similar way as you use them in normal SQL SELECT query.

The WITH CHECK OPTION:

The WITH CHECK OPTION is a CREATE VIEW statement option. The purpose of the WITH CHECK OPTION is to ensure that all UPDATE and INSERTs satisfy the condition(s) in the view definition. If they do not satisfy the condition(s), the UPDATE or INSERT returns an error.

Example:

```
CREATE VIEW CUSTOMERS_VIEW AS
SELECT name, age
FROM CUSTOMERS
WHERE age IS NOT NULL
WITH CHECK OPTION;
```

The WITH CHECK OPTION in this case should deny the entry of any NULL values in the view's AGE column, because the view is defined by data that does not have a NULL value in the AGE column. Updating a View:

A view can be updated under certain conditions:

- The SELECT clause may not contain the keyword DISTINCT.
- The SELECT clause may not contain summary functions.
- The SELECT clause may not contain set functions.
- The SELECT clause may not contain set operators.
- The SELECT clause may not contain an ORDER BY clause.
- The FROM clause may not contain multiple tables.
- The WHERE clause may not contain subqueries.
- The query may not contain GROUP BY or HAVING.
- Calculated columns may not be updated.
- All NOT NULL columns from the base table must be included in the view in order for the INSERT query to function.

So if a view satisfies all the above mentioned rules then you can update a view. Following is an example to update the age of Ramesh:

```
SQL > UPDATE CUSTOMERS_VIEW
SET AGE = 35
WHERE name='Ramesh';
```

This would ultimately update the base table CUSTOMERS and same would reflect in the view itself. Now, try to query base table, and SELECT statement would produce the following result:

SQL CREATE VIEW Examples-The following SQL creates a view that shows all customers from Kathmandu:

Example

```
CREATE VIEW [Kathmandu Customers] AS
SELECT CustomerName, ContactName
FROM Customers
WHERE city = 'Kathmandu';
```

We can query the view above as follows:

Example

```
SELECT * FROM [Kathmandu Customers];
```

The following SQL creates a view that selects every product in the "Products" table with a price higher than the average price:

Example

```
CREATE VIEW [Products Above Average Price] AS
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products);
```

We can query the view above as follows:

Example

```
SELECT * FROM [Products Above Average Price];
```

SQL Updating a View

A view can be updated with the CREATE OR REPLACE VIEW statement.

SQL CREATE OR REPLACE VIEW Syntax

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

The following SQL adds the "City" column to the "Kathmandu Customers" view:

Example

```
CREATE OR REPLACE VIEW [Kathmandu Customers] AS
SELECT CustomerName, ContactName, City
FROM Customers
WHERE City = 'Brazil';
```

SQL Dropping a View

A view is deleted with the DROP VIEW statement.

SQL DROP VIEW Syntax

```
DROP VIEW view_name;
```

The following SQL drops the "Brazil Customers" view:

Example

```
DROP VIEW [Kathmandu Customers];
```

Lab Task: 5

1. Perform join operation on teacher and employee table and display the Ename , Faculty and salary.
2. Perform left join on table book list and book table.
3. Perform right join on booklist and book table.
4. Perform Full join on student and issues table.
5. Display those employees name and salary whose name starts with 's' and whose name consist 'a' as sub string.
6. Display name of the employee who is also a teacher.
7. Display all employees name except the name who are teachers.
8. Create a view Employee-view which consist of eid, ename , salary as attributes.
9. Insert a new record in recently created view. And also display the contents of primary table.
10. Delete the information from view where salary are less than 29000.

Stored Procedures & Inbuilt Functions

What is a Stored Procedure?

A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again. So if you have an SQL query that you write over and over again, save it as a stored procedure, and then just call it to execute it. You can also pass parameters to a stored procedure, so that the stored procedure can act based on the parameter value(s) that is passed.

Stored Procedure Syntax

```
CREATE PROCEDURE procedure_name
AS
sql_statement
GO;
```

Execute a Stored Procedure

```
EXEC procedure_name;
```

How to create a Procedure?

Syntax:

```
CREATE PROCEDURE procedure_name
AS
Sql_expression GO
```

Example:

Example:

```
create procedure show1 as
select * from example go
```

How to run the procedure?

```
Exec procedure_name
OR
Procedure_name
```

When creating a stored procedure, you can either use CREATE PROCEDURE or CREATE PROC. After the stored procedure name, you need to use the keyword "AS" and then the rest is just the regular SQL code that you would normally execute.

One thing to note is that you cannot use the keyword "GO" in the stored procedure. Once the SQL Server compiler sees "GO" it assumes it is the end of the batch.

Also, you cannot change database context within the stored procedure such as using "USE dbName" the reason for this is because this would be a separate batch and a stored procedure is a collection of only one batch of statements.

How to create a SQL Server stored procedure with parameters:

The real power of stored procedures is the ability to pass parameters and have the stored procedure handle the differing requests that are made. In this topic we will look at passing parameter values to a stored procedure.

Explanation

Just like you have the ability to use parameters with your SQL code you can also setup your stored procedures to accept one or more parameter values.

Stored Procedure With One Parameter

The following SQL statement creates a stored procedure that selects Customers from a particular City from the "Customers" table:

Example

```
CREATE PROCEDURE SelectAllCustomers @City nvarchar(30)
AS
```

```
SELECT * FROM Customers WHERE City = @City
GO;
```

Execute the stored procedure above as follows:

Example

```
EXEC SelectAllCustomers @City = 'London';
```

Stored Procedure With Multiple Parameters

Setting up multiple parameters is very easy. Just list each parameter and the data type separated by a comma as shown below.

The following SQL statement creates a stored procedure that selects Customers from a particular City with a particular PostalCode from the "Customers" table:

Example

```
CREATE PROCEDURE SelectAllCustomers @City nvarchar(30), @PostalCode nvarchar(10)
AS
SELECT * FROM Customers WHERE City = @City AND PostalCode = @PostalCode
GO;
```

Execute the stored procedure above as follows:

Example

```
EXEC SelectAllCustomers @City = 'London', @PostalCode = 'WA1 1DP';
```

Default Parameter Values

In most cases it is always a good practice to pass in all parameter values, but sometimes it is not possible. So in this example we use the NULL option to allow you to not pass in a parameter value. If we create and run this stored procedure as is it will not return any data, because it is looking for any City values that equal NULL.

```
CREATE PROCEDURE uspGetAddress @City nvarchar(30) = NULL
AS
SELECT *
FROM AdventureWorks.Person.Address
WHERE City = @City
GO
```

We could change this stored procedure and use the ISNULL function to get around this. So if a value is passed it will use the value to narrow the result set and if a value is not passed it will return all records. (Note: if the City column has NULL values this will not include these values. You will have to add additional logic for City IS NULL)

```
CREATE PROCEDURE uspGetAddress @City nvarchar(30) = NULL
AS
SELECT *
FROM AdventureWorks.Person.Address
WHERE City =
ISNULL(@City, City) GO
```

Multiple Parameters

Setting up multiple parameters is very easy to do. You just need to list each parameter and the data type separated by a comma as shown below.

```
CREATE PROCEDURE uspGetAddress @City nvarchar(30) = NULL, @AddressLine1
nvarchar(60) = NULL
AS
SELECT *
```

```
FROM AdventureWorks.Person.Address
WHERE City = ISNULL(@City, City)
AND AddressLine1 LIKE '%' + ISNULL(@AddressLine1
, AddressLine1) + '%' GO
```

To execute this you could do any of the following:

```
EXEC uspGetAddress @City = 'Calgary'
--or
EXEC uspGetAddress @City = 'Calgary', @AddressLine1 = 'A'
--or
EXEC uspGetAddress @AddressLine1
= 'Acadia' -- etc...
```

Example:

```
CREATE PROCEDURE show4 @nam
varchar(30), @idi int AS
SELECT *
FROM example
WHERE name like @nam
+'%' or id=@idi GO
drop procedure
show4 show4
@nam
='s', @idi=12
```

Deleting a SQL Server stored procedure

Dropping Single Stored Procedure

To drop a single stored procedure you use the DROP PROCEDURE or DROP PROC command as follows.

```
DROP PROCEDURE uspGetAddress
GO
-- or
DROP PROC uspGetAddress
GO
-- or
DROP PROC dbo.uspGetAddress -- also specify the schema
```

Dropping Multiple Stored Procedures

To drop multiple stored procedures with one command you specify each procedure separated by a comma as shown below.

```
DROP PROCEDURE uspGetAddress, uspInsertAddress,
uspDeleteAddress GO
-- or
DROP PROC uspGetAddress, uspInsertAddress, uspDeleteAddress
GO
```

Naming conventions for SQL Server stored Procedures

SQL Server uses object names and schema names to find a particular object that it needs to work with. This could be a table, stored procedure, function ,etc...

It is a good practice to come up with a standard naming convention for you objects including stored procedures.

Do not use sp_ as a prefix

One of the things you do not want to use as a standard is "sp_". This is a standard naming convention that is used in the master database. If you do not specify the database where the object is, SQL Server will first search the master database to see if the object exists there and then it will search the user database. So avoid using this as a naming convention.

Standardize on a Prefix

It is a good idea to come up with a standard prefix to use for your stored procedures. As mentioned above do not use "sp_", so here are some other options.

- usp_
- sp
- usp
- etc...

To be honest it does not really matter what you use. SQL Server will figure out that it is a stored procedure, but it is helpful to differentiate the objects, so it is easier to manage.

So a few examples could be:

- spInsertPerson
- uspInsertPerson
- usp_InsertPerson
- InsertPerson

Again this is totally up to you, but some standard is better than none.

Naming Stored Procedure Action

Based on the actions that you may take with a stored procedure, you may use:

- Insert
- Delete
- Update
- Select
- Get
- Validate □ etc...

So here are a few examples:

- uspInsertPerson
- uspGetPerson
- spValidatePerson □ SelectPerson □ etc...

Another option is to put the object name first and the action second, this way all of the stored procedures for an object will be together.

- uspPersonInsert
- uspPersonDelete □ uspPersonGet □ etc...

Again, this does not really matter what action words that you use, but this will be helpful to classify the behavior characteristics.

Naming Stored Procedure Object

Keep the names simple, but meaningful. As your database grows and you add more and more objects you will be glad that you created some standards.

So some of these may be:

- uspInsertPerson - insert a new person record
- uspGetAccountBalance - get the balance of an account
- uspGetOrderHistory - return list of orders

Using comments in a SQL Server stored procedure

SQL Server offers two types of comments in a stored procedure; line comments and block comments. The following examples show you how to add comments using both techniques. Comments are displayed in green in a SQL Server query window.

Line Comments

To create line comments you just use two dashes "--" in front of the code you want to comment. You can comment out one or multiple lines with this technique.

In this example the entire line is commented out.

-- this procedure gets a list of addresses based

-- on the city value that is passed

```
CREATE PROCEDURE uspGetAddress @City nvarchar(30)
```

```
AS
```

```
SELECT *
```

```
FROM AdventureWorks.Person.Address
```

```
WHERE City =
```

```
@City GO
```

This next example shows you how to put the comment on the same line.

-- this procedure gets a list of addresses based on the city value that is passed

```
CREATE PROCEDURE uspGetAddress @City nvarchar(30)
```

```
AS
```

```
SELECT *
```

```
FROM AdventureWorks.Person.Address
```

```
WHERE City = @City -- the @City parameter value will narrow the search criteria GO
```

Block Comments

To create block comments the block is started with "/*" and ends with "*/". Anything within that block will be a comment section.

```
/*
```

```
-this procedure gets a list of addresses based  
on the city value that is passed
```

```
-this procedure is used by the HR system */
```

```
CREATE PROCEDURE uspGetAddress @City
```

```
nvarchar(30) AS
```

```
SELECT *
```

```
FROM AdventureWorks.Person.Address
```

```
WHERE City = @City
```

```
GO
```

Combining Line and Block Comments

You can also use both types of comments within a stored procedure.

```
/*
```

```
-this procedure gets a list of addresses based  
on the city value that is passed
```

```
-this procedure is used by the HR system */
```

```
CREATE PROCEDURE uspGetAddress @City nvarchar(30)
```

```
AS
```

```
SELECT *
```

```
FROM AdventureWorks.Person.Address
```

```
WHERE City = @City -- the @City parameter value will narrow the search criteria GO
```

Returning stored procedure parameter values to a calling stored procedure

In a previous topic we discussed how to pass parameters into a stored procedure, but another option is to pass parameter values back out from a stored procedure. One option for this may be that you call another stored procedure that does not return any data, but returns parameter values to be used by the calling stored procedure.

Explanation

Setting up output parameters for a stored procedure is basically the same as setting up input parameters, the only difference is that you use the OUTPUT clause after the parameter name to specify that it should return a value. The output clause can be specified by either using the keyword "OUTPUT" or just "OUT".

Simple Output

```
CREATE PROCEDURE uspGetAddressCount @City nvarchar(30),
@AddressCount int OUTPUT AS
SELECT @AddressCount = count(*)
FROM
```

```
AdventureWorks.Person.Address
```

```
WHERE City = @City
```

Or it can be done this way:

```
CREATE PROCEDURE uspGetAddressCount @City nvarchar(30),
@AddressCount int OUT AS
SELECT @AddressCount = count(*)
FROM AdventureWorks.Person.Address
```

```
WHERE City = @City
```

To call this stored procedure we would execute it as follows. First we are going to declare a variable, execute the stored procedure and then select the returned value.

```
DECLARE @AddressCount int
EXEC uspGetAddressCount @City = 'Calgary', @AddressCount = @AddressCount
OUTPUT SELECT @AddressCount
```

This can also be done as follows, where the stored procedure parameter names are not passed.

```
DECLARE @AddressCount int
EXEC uspGetAddressCount 'Calgary', @AddressCount OUTPUT
SELECT @AddressCount
```

Modifying an existing SQL Server stored procedure

Modifying or ALTERing a stored procedure is pretty simple. Once a stored procedure has been created it is stored within one of the system tables in the database that it was created in. When you modify a stored procedure the entry that was originally made in the system table is replaced by this new code. Also, SQL Server will recompile the stored procedure the next time it is run, so your users are using the new logic. The command to modify an existing stored procedure is ALTER PROCEDURE or ALTER PROC.

Modifying an Existing Stored Procedure

Let's say we have the following existing stored procedure: This allows us to do an exact match on the City.

```
CREATE PROCEDURE uspGetAddress @City nvarchar(30)
AS
SELECT *
FROM AdventureWorks.Person.Address
```

WHERE City =

@City GO

Let's say we want to change this to do a LIKE instead of an equals.

To change the stored procedure and save the updated code you would use the ALTER PROCEDURE command as follows.

```
ALTER PROCEDURE uspGetAddress @City nvarchar(30)
```

```
AS
```

```
SELECT *
```

```
FROM AdventureWorks.Person.Address
```

```
WHERE City LIKE @City +
```

```
'%' GO
```

Now the next time that the stored procedure is called by an end user it will use this new logic.

Sql date time functions:

- [DateTime Function in SQL Server](#)
- [GETDATE\(\)](#)
- [DATEPART\(\)](#)
- [DATEDIFF\(\)](#)
- [DATENAME\(\)](#)
- [DAY\(\)](#)
- [MONTH\(\)](#) □ [YEAR\(\)](#)

GETDATE() is very common used method which returns exact date time from the system. It does not accept any parameter. Just call it like simple function.

Example:

```
SELECT getdate()
```

DATEADD()

DATEADD() is used to add or subtract datetime. Its return a new datetime based on the added or subtracted interval.

General Syntax

```
DATEADD(datepart, number, date)
```

datepart is the parameter that specifies on which part of the date to return a new value. Number parameter is used to increment datepart.

Example:

```
SELECT getdate();
```

```
SELECT DATEADD(day, 5,getdate() ) AS
```

NewTime Output:

```
2015-08-12 09:18:18.080
```

```
2015-08-17 09:18:18.080
```

DATEPART()

DATEPART() is used when we need a part of date or time from a datetime variable. We can use DATEPART() method only with select command.

Syntax

```
DATEPART(datepart, date)
```

Example:

```
SELECT DATEPART(year, GETDATE()) AS 'Year'
```

```
SELECT DATEPART(hour, GETDATE()) AS 'Hour'
```

DATEDIFF()

DATEDIFF() is very common function to find out the difference between two DateTime elements.

Syntax

DATEDIFF(datepart, startdate, enddate)

Example:

```
SELECT DATEDIFF(day, @Date1, @Date2) AS
```

DifferenceOfDay

DATENAME()

DATENAME() is very common and most useful function to find out the date name from the datetime value.

-- Get Today

```
SELECT DATENAME(dw, getdate()) AS 'Today Is'
```

-- Get Mont name

```
SELECT DATENAME(month, getdate()) AS
```

'Month'

DAY()

DAY() is used to get the day from any date time object.

Example:

```
SELECT DAY(getdate()) AS 'DAY'
```

Output :

DAY

12

MONTH()

```
SELECT MONTH(getdate()) AS 'Month'
```

YEAR()

```
SELECT YEAR(getdate()) AS 'Year'
```

String Functions

- [ASCII\(\)](#)
- [CHAR\(\)](#)
- [LEFT\(\)](#)
- [RIGHT\(\)](#)
- [LTRIM\(\)](#)
- [RTRIM\(\)](#)
- [REPLACE\(\)](#)
- [QUOTENAME\(\)](#)
- [REVERSE\(\)](#)
- [CHARINDEX\(\)](#)
- [PATINDEX\(\)](#)
- [LEN\(\)](#)
- [STUFF\(\)](#)
- [SUBSTRING\(\)](#)
- [LOWER/UPPER\(\)](#)

ASCII()

Returns the ASCII code value of the leftmost character of a character expression.

Syntax ASCII (CHARACTER_EXPRESSION)

```
SELECT ASCII('K'); returns 107
```

CHAR()

Converts an **int** ASCII code to a character.

Syntax

CHAR (*integer_expression*)

Arguments: integer_expression: Is an integer from 0 through 255. NULL is returned if the integer expression is not in this range.

LEFT()

Returns the left most characters of a string.

Syntax

LEFT(*string*, *length*)

string

Specifies the string from which to obtain the left-most characters.

length

Specifies the number of characters to obtain.

Example:

```
SELECT LEFT('kathmandu',5)
```

Output:

```
Kathm
```

RIGHT()

Returns the right most characters of a string.

Syntax

RIGHT(*string*, *length*)

string

Specifies the string from which to obtain the left-most characters.

length

Specifies the number of characters to obtain.

LTRIM()

Returns a character expression after it removes leading blanks.

Example :

```
SELECT LTRIM(' Md. Ashok')
```

Output :

```
Md. Ashok
```

RTRIM()

Returns a character string after truncating all trailing blanks.

Example :

```
SELECT RTRIM('Md. barsha  ')
```

Output:

```
Md. Barsha
```

REVERSE()

Returns a character expression in reverse order.

Example :

```
SELECT REVERSE(ename) FROM Employee;
```

CHARINDEX

CharIndex returns the first occurrence of a string or characters within another string. The Format of CharIndex is given Below:

CHARINDEX (expression1 , expression2 [, start_location])

Here *expression1* is the string of characters to be found within *expression2*. So if you want to search *ij* within the word *Abhijit*, we will use *ij* as *expression1* and *Abhijit* as *expression2*. start location is an optional integer argument which identifies the position from where the string will be searched. Now let us look into some examples :

```
SELECT CHARINDEX('rma', 'Ramesh Sharma', 3) AS MatchPosition;
SELECT CHARINDEX('SQL', 'Microsoft SQL Server')
```

OUTPUT:

11

So it will start from 1 and go on searching until it finds the total string element searched, and returns its first position. The Result will be 0 if the searched string is not found.

We can also mention the Start_Location of the string to be searched.

EXAMPLE:

```
SELECT CHARINDEX('SQL', 'Microsoft SQL server has a great SQL Engine',12)
```

So in the above example we can have the Output as 34 as we specified the StartLocation as 12, which is greater than initial SQL position(11).

PATINDEX

As a contrast Patindex is used to search a pattern within an expression. The Difference between CharIndex and PatIndex is the later allows WildCard Characters.

PATINDEX ('%pattern%' , expression)

Here the first argument takes a pattern with wildcard characters like '%' (meaning any string) or '_' (meaning any character).

For Example

```
SELECT PATINDEX('%ani%', 'ShyamManiAcharya');
```

Output:

7

LEN

Len is a function which returns the length of a string. This is the most common and simplest function that everyone use. Len Function excludes trailing blank spaces.

```
SELECT LEN('ABHISHEK IS WRITING THIS')
```

This will output 24, it is same when we write `LEN('ABHISHEK IS WRITING THIS ')` as LEN doesnt take trailing spaces in count.

SUBSTRING

Substring returns the part of the string from a given character expression. The general syntax of Substring is as follows :

SUBSTRING(expression, start, length)

Here the function gets the string from start to length. Let us take an example below:

SELECT OUT = SUBSTRING('abcdefgh', 2, 3)

The output will be "bcd".

Note : substring also works on ntext, VARCHAR, CHAR etc.

LOWER / UPPER

Another simple but handy function is Lower / UPPER. The will just change case of a string expression. For Example,

SELECT UPPER('this is Lower TEXT')

SQL CREATE VIEW Statement

In SQL, a view is a virtual table based on the result-set of an SQL statement. A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database. You can add SQL statements and functions to a view and present the data as if the data were coming from one single table. A view is created with the CREATE VIEW statement.

Syntax

```
create view view_name as
select column1, column2, ...
from table_name
where condition;
```

Example

```
create view [above average] as
select book.bid,book.bname, price
from book
where price > (select avg(price) from book);
```

Using views by

```
select * from [Above Average];
```

	bid	bname	Price
1	2001	DBMS	500
2	2004	Data Mining	600

CREATE PROCEDURE

The CREATE PROCEDURE command is used to create a stored procedure. A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again.

Example

```
create procedure books
as
select * from book
go;
```

Using procedure as follows:

```
exec books
```